MineC: uma linguagem de domínio específico para irrigação automática

Novembro de 2026

Autores: Bernardo Moreira Guimarães Gonçalves, Guilherme Dias Batista, Hugo Watzl, Eduardo Jacob, Lucca Barcelos. Disciplina de Linguagens Formais e Compiladores.

Professor: Willian de Souza Barbosa.

Resumo

Este artigo apresenta o MineC, uma linguagem imperativa de domínio específico para programar a irrigação de uma plantação, com vocabulário inspirado no jogo Minecraft. O compilador implementa as análises léxica, sintática e semântica, e um interpretador executa a árvore sintática gerada. Um painel web e um firmware para ESP32 completam o sistema, de modo que toda a lógica da fazenda fica escrita na própria linguagem.

A gramática foi verificada como LL(1) por uma ferramenta que calcula os conjuntos FIRST e FOLLOW, sem nenhum conflito. Em 6000 programas de teste, o validador da gramática e o parser concordaram em todos os casos. A suíte tem 125 testes automatizados, e 35 de 37 defeitos introduzidos de propósito no servidor foram detectados.

O sistema foi validado em simulação e com uma placa imitada por software. Os testes em bancada com sensor, relé e bomba reais ainda não foram feitos e são a principal limitação deste rascunho.

Palavras-chave: compiladores; linguagens de domínio específico; gramática LL(1); análise semântica; irrigação automática; ESP32.

1. Introdução

Uma plantação automática repete sempre o mesmo ciclo: ler a umidade do solo, decidir se precisa regar e acionar a bomba. Essa lógica costuma ficar escondida no firmware do microcontrolador, onde mudar uma regra exige gravar a placa de novo. Escrevê-la numa linguagem própria torna a regra legível e alterável sem tocar no hardware.

Linguagens de domínio específico trocam generalidade por expressão direta dos conceitos de um problema [1]. No MineC, sensor e atuador são conceitos da própria linguagem, com palavras reservadas e regras semânticas próprias. O vocabulário do Minecraft reduz a barreira de entrada e mantém o projeto reconhecível para quem conhece o jogo.

O objetivo do trabalho é percorrer as fases de um compilador num caso completo, do texto do programa até o acionamento de uma bomba. A Seção 2 apresenta a linguagem, a Seção 3 descreve o compilador e a Seção 4 mostra a execução e a arquitetura. As Seções 5 e 6 tratam de segurança e verificação, e as Seções 7 e 8 discutem limitações e concluem.

2. A linguagem MineC

O MineC é imperativo, sem terminador de instrução, com blocos entre chaves, comentários com `#` e sensível a maiúsculas. Tem quatro tipos de valor: inteiro, decimal, texto e booleano. O controle de fluxo usa `if`, `else if`, `else`, `while` e `repeat`, e funções são declaradas com `command` apenas no nível global.

Conceito	Palavra	Efeito no protótipo
Variável	`craft`	guarda um valor
Sensor	`observer`	associa um nome ao pino analógico do ESP32
Atuador	`lever`	associa um nome ao pino do relé
Ler sensor	`mine`	atualiza o valor com a umidade em %, ou -1 sem leitura confiável
Ligar e desligar	`power`, `unpower`	aciona o relé da bomba
Função e retorno	`command`, `drop`	agrupa instruções e devolve um valor
Saída	`say`	escreve no console do painel
Espera	`wait`	pausa por um tempo em segundos

O programa abaixo foi gerado pelo painel para um canteiro e mostra o ciclo completo da fazenda.

```
observer cenouras = 34
lever bomba = 26

craft intervalo = 120
craft tempoRega = 10
craft minimoCenouras = 30

command regar(segundos) {
    power bomba
    wait segundos
    unpower bomba
}
```

```
while true {
    mine cenouras
    if cenouras < 0 {
        say "sensor sem leitura válida, rega suspensa"
    } else if cenouras < minimoCenouras {
        say "solo seco (" + cenouras + "%), irrigando"
        regar(tempoRega)
    } else {
        say "umidade ok (" + cenouras + "%)"
    }
    wait intervalo
}
```

3. O compilador

O compilador transforma o texto em tokens, os tokens em árvore sintática e valida essa árvore antes de qualquer execução. As fases seguem a organização clássica descrita em [2]. Um erro em qualquer fase impede o programa de rodar e aponta a linha do problema.

3.1 Análise léxica

O lexer é um autômato finito determinístico escrito à mão, que reconhece 39 tipos de token além do fim de arquivo [3]. Ele segue a regra do maior casamento: `<=` é um token só, nunca `<` seguido de `=`. Palavras reservadas são lidas como identificadores e depois consultadas numa tabela, o que simplifica o autômato.

```
TK_IDENTIFIER  letra (letra | digito)*
TK_INTEGER     digito+
TK_DECIMAL     digito+ "." digito+
TK_STRING      "\"" [^"\n]* "\""
comentario     "#" [^\n]*
```

3.2 Análise sintática

A gramática livre de contexto foi escrita para ser LL(1), e o parser usa descida recursiva sem retrocesso [2, 4]. Como nenhuma instrução começa com `-` ou `(`, um único token de lookahead basta para separar instruções, e o `;` se torna desnecessário. O uso de chaves elimina a ambiguidade do `else` pendente.

A precedência fica codificada na cascata de regras, da menor para a maior, e os níveis binários associam à esquerda.

```
Expr    → Conj { or Conj }
Conj    → Igual { and Igual }
Igual   → Rel { (== | !=) Rel }
Rel     → Soma { (< | > | <= | >=) Soma }
Soma    → Termo { (+ | -) Termo }
Termo   → Unario { (* | /) Unario }
Unario  → (not | -) Unario | Primario
```

3.3 Análise semântica

A fase semântica percorre a árvore com uma tabela de símbolos encadeada por escopo. Ela verifica declaração antes do uso, compatibilidade de tipos, quantidade de argumentos, `drop` apenas dentro de `command` e `mine` e `power` apenas nos alvos corretos.

O domínio acrescenta regras que uma linguagem genérica não teria, e o compilador conhece os pinos do ESP32:

- Sensor analógico só em ADC1, porque os pinos ADC2 não leem com o Wi-Fi ligado [5].
- Atuador nunca nos pinos de memória flash (6 a 11) nem nos somente entrada (34 a 39).
- Um mesmo pino não pode ser sensor e atuador ao mesmo tempo.

Em modo ESP32 essas violações são erros; em simulação, são avisos. Assim, um pino errado é recusado antes de chegar à placa.

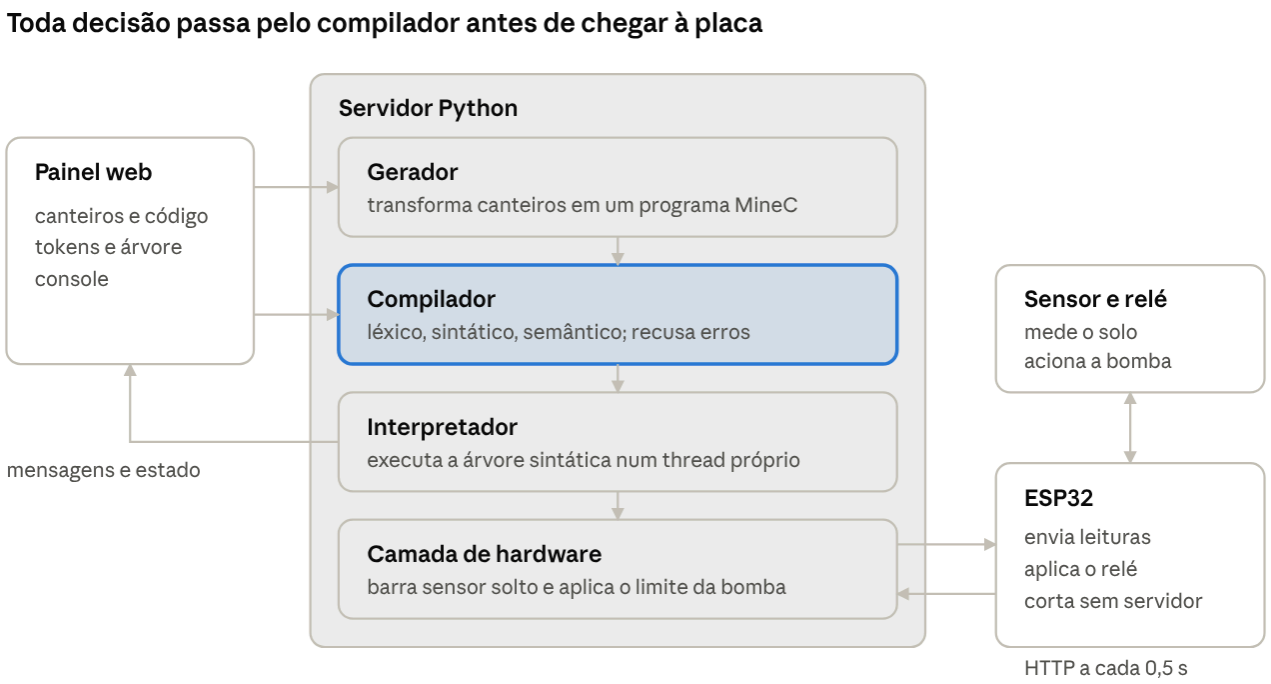
3.4 Tratamento de erros

O compilador nunca lança exceção por causa do conteúdo do código: ele sempre devolve uma lista de mensagens com fase e linha. Entradas extremas, como 500 parênteses aninhados, também viram erro com orientação de correção. Literais têm limite: inteiro até 4000 dígitos e texto até 100.000 caracteres.

Além dos erros, o compilador gera avisos para código válido mas suspeito. Exemplos são um `while` sempre verdadeiro sem `wait` e um sensor usado sem nenhum `mine` no programa.

4. Execução e arquitetura

Toda decisão da fazenda passa pelo compilador: o painel não aciona o hardware direto, ele gera um programa MineC. O ESP32 não guarda lógica, apenas lê sensores e aplica o estado que o servidor manda. Trocar a regra de irrigação, portanto, não exige regravar a placa.



Arquitetura do MineC: do painel à bomba

O painel envia canteiros ao gerador ou código direto ao compilador; o que não compila não chega à execução.

Gerador. Cada canteiro cadastrado no painel vira declarações e um trecho do laço principal. O gerador garante que nenhum nome dado pelo usuário colida com os nomes internos do programa.

Interpretador. Percorre a árvore em um thread separado, com ambientes encadeados para o escopo. Ele acessa o mundo físico apenas por `ler_sensor` e `acionar`, de modo que o mesmo programa roda em simulação ou no ESP32. Um `wait` é descontado em fatias de 0,2 s, o que permite parar o programa quase na hora.

Servidor. Escrito em Python, usa apenas a biblioteca padrão. Serve o painel, guarda a configuração e só salva um programa novo depois que ele compila.

Painel. Em HTML, CSS e JavaScript puros, mostra a umidade de cada canteiro e permite editar o código. As abas de tokens e de árvore sintática exibem as fases do compilador sobre o programa atual, o que também serve de material didático.

Firmware. A cada 0,5 s, o ESP32 envia as leituras brutas por HTTP e recebe o estado desejado de cada relé. O protocolo é texto simples, com uma chave e um valor por linha.

```
POST /api/esp32/sync
id=minec-esp32-01
rssi=-58
a34=2710

resposta:
ler=34
saidas=26:1
periodo=500
limite=120
```

5. Segurança

Uma bomba presa ligada alaga o canteiro, então nenhuma camada confia sozinha nas outras. O firmware segue funcionando se o servidor sumir, e o servidor recusa decisões baseadas em dados ruins. A tabela resume cada falha considerada, a resposta e como ela foi conferida até aqui.

Falha	Resposta do sistema	Onde	Conferido por
Sensor solto (leitura bruta até 50 ou a partir de 4090)	`mine` devolve -1 e a bomba daquele sensor não liga	Servidor	Teste com placa falsa
Bomba ligada além do limite (padrão 120 s)	Corta e só religa depois de um `unpower`	Servidor	Teste com placa falsa
Mesma falha, sem o servidor	Corta e só religa depois de o servidor pedir para desligar	Firmware	Leitura do código
Servidor ou Wi-Fi fora do ar	Desliga os relés em até 5 s	Firmware	Cálculo, ainda sem medição
Programa inválido	Não substitui o programa salvo	Servidor	Teste automatizado
Página de outro site comandando a bomba	Recusa pedido de outra origem	Servidor	Teste em navegador real

O prazo de 5 s do firmware vem de uma soma: 1,8 s de silêncio tolerável, até 3 s de bloqueio da chamada HTTP e 0,2 s de folga. A leitura do código-fonte da biblioteca mostrou que `getString()` pode travar sem prazo, por isso o firmware confere a chegada do corpo antes de chamá-la.

Dois riscos seguem abertos. O painel não tem login, então quem está na mesma rede pode usá-lo, decisão aceita para um protótipo de sala de aula. E um programa personalizado que alterna `power` e `unpower` sem parar não é barrado, porque o limite conta cada ligação separadamente.

6. Verificação e resultados

O software passou em todas as verificações automatizadas, mas nada foi testado ainda com hardware real. A verificação combinou testes de unidade, testes contra o servidor em execução, fuzz e testes no navegador. Para saber se os testes medem algo de fato, defeitos foram introduzidos de propósito no código.

Verificação	O que foi feito	Resultado
Conflitos LL(1)	Tabela preditiva calculada a partir da gramática	Nenhum conflito
Testes do compilador, interpretador e gerador	81 testes	Todos passam
Testes do servidor por HTTP	44 testes com o servidor real e uma placa imitada	Todos passam
Gramática x parser	6000 programas gerados e aleatórios	0 divergências
Execução com fuzz	1981 programas válidos, com leituras 55, -1, 0 e 100	Nenhuma exceção Python vazou
Nomes de canteiro	4000 configurações aleatórias no gerador	Todas compilam
Defeitos no servidor e no hardware	37 sabotagens, cada uma com o teste esperado	35 detectadas
Defeitos no compilador	47 sabotagens, conforme o script do projeto	Todas detectadas após reforçar 2 testes
Painel no navegador	32 verificações de ponta a ponta	Todas passam

A sabotagem também mediu a qualidade dos testes. Um deles passava mesmo com a regra desligada e foi reforçado. Das duas sabotagens no servidor que seguem sem detecção, uma atinge uma proteção que nenhum caminho conhecido alcança e a outra só abre uma condição de corrida rara.

O teste exploratório inicial encontrou 18 problemas que a suíte original, de 28 testes verdes, não via. Os mais graves eram um sensor solto acionando a bomba a cada ciclo e um programa com erro salvo antes de compilar. Todos foram corrigidos, com exceção das pendências de hardware.

Pendente: medir o sistema real. Os dados de bancada (tempo de corte, calibração do sensor, soltura do relé) devem entrar nesta seção quando existirem.

7. Discussão e limitações

O MineC é interpretado no servidor, não compilado para código de máquina. A escolha simplifica mudanças de programa e mantém o ESP32 pequeno, mas torna o sistema dependente da rede: sem servidor, a fazenda só desliga as bombas, e não rega. Gerar código para a placa removeria essa dependência.

Outras limitações conhecidas:

- **Erros:** o lexer e o parser param no primeiro erro, sem recuperação em modo pânico.
- **Tipos:** o valor devolvido por um `command` só é conhecido na execução, então parte dos erros de tipo aparece tarde.
- **Valor -1:** um programa escrito à mão precisa tratar o -1 do `mine`. Sem isso, ele pode anunciar uma rega que a trava do servidor impede.
- **Validação:** a suíte usa uma placa imitada, e os testes de firmware dependem de bancada. Firmware, sensor, relé e rede real não foram exercitados.
- **Compilação lenta:** uma soma com 992 termos levou 3,6 s entre compilar e serializar o resultado. Não afeta programas normais.

8. Conclusão e trabalhos futuros

O MineC mostra as fases de um compilador num problema concreto, com regras semânticas que só fazem sentido naquele domínio, como os pinos válidos do ESP32. Toda a lógica de irrigação fica escrita na linguagem e pode ser lida nas abas do próprio painel. A verificação em software foi extensa, e o passo seguinte é validar o sistema com hardware real.

Trabalhos futuros, em ordem de prioridade:

1. Testar em bancada o corte de segurança, a calibração e o relé, e registrar os resultados.
2. Implementar recuperação de erros em modo pânico, para relatar vários erros por compilação.
3. Gerar código para o ESP32, permitindo que a fazenda funcione sem servidor.

4. Limitar o tempo total de bomba ligada por período, para cobrir programas que alternam liga e desliga.

Referências

1. MERNIK, M.; HEERING, J.; SLOANE, A. M. When and how to develop domain-specific languages. *ACM Computing Surveys*, v. 37, n. 4, p. 316-344, 2005. DOI: 10.1145/1118890.1118892.
2. AHO, A. V.; LAM, M. S.; SETHI, R.; ULLMAN, J. D. *Compilers: Principles, Techniques, and Tools*. 2. ed. Boston: Pearson/Addison-Wesley, 2007.
3. HOPCROFT, J. E.; MOTWANI, R.; ULLMAN, J. D. *Introduction to Automata Theory, Languages, and Computation*. 3. ed. Boston: Pearson/Addison-Wesley, 2007.
4. LEWIS, P. M.; STEARNS, R. E. Syntax-directed transduction. *Journal of the ACM*, v. 15, n. 3, p. 465-488, 1968. DOI: 10.1145/321466.321477.
5. ESPRESSIF SYSTEMS. *ESP-IDF Programming Guide: Analog to Digital Converter (ADC)*. Disponível em: docs.espressif.com. Acesso em: 5 nov. 2026.

Repositórios do projeto: compilador e especificação e protótipo.